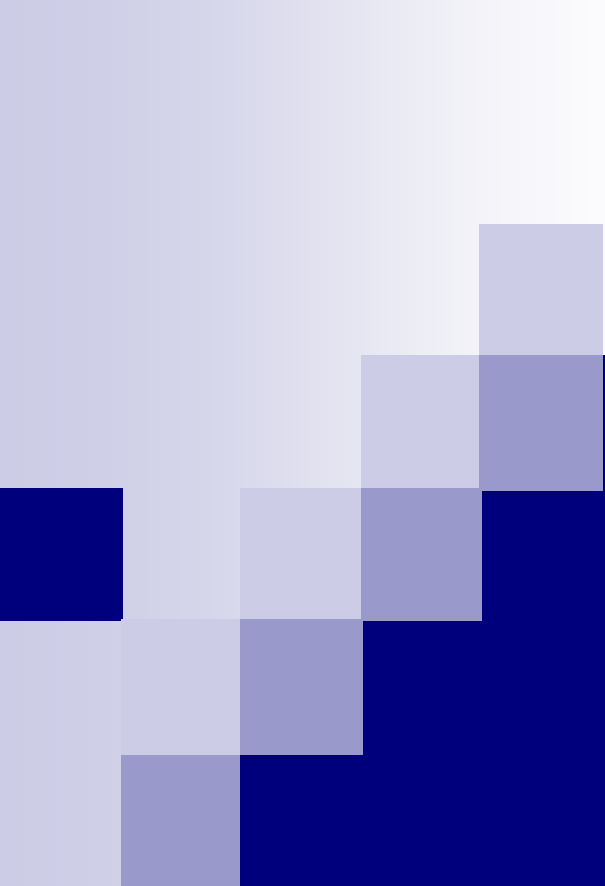




Organização de Computadores 1

1 - INTRODUÇÃO

Prof. Luiz Gustavo A. Martins

Arquitetura

- ★ Define os elementos que impactuam diretamente na execução lógica do programa.
- ★ Corresponde aos atributos **visíveis** aos programadores.
- ★ Ex: conj. de instruções, nº de bits usados para representar dados, técnicas de endereçamento.
 - ✓ Há uma instrução para multiplicação?

Organização

- ✦ Refere-se aos aspectos de implantação não visíveis ao programador.
- ✦ Define como os elementos da arquitetura são estruturados na máquina.
- ✦ Ex: sinais de controle, interfaces, tecnologia de memória.
 - ✓ Como a multiplicação é implementada (HW específico ou sucessivas somas)?



Organização e Arquitetura

- ★ Famílias de computadores costumam compartilhar a mesma arquitetura básica.
 - ✓ Compatibilidade/reaproveitamento de código
 - ✗ Pelo menos para as versões anteriores.
 - ✓ Ex: Intel x86 e IBM System/370
- ★ A organização difere entre diferentes modelos.
 - ✓ Afeta custo e desempenho

Organização Estruturada (Multi-níveis)

✦ Ponto de vista de engenharia:

- ✓ Computador é um sistema complexo.
 - ✦ Formado por milhões de componentes eletrônicos elementares
- ✓ **Desafio:** Definir e projetar este sistema de forma estruturada e sistemática.
- ✓ **Solução:** Realizar uma série de abstrações do computador organizadas em níveis de hierarquia.
 - ✦ Cada nível é composto por um conjunto de componentes e seus relacionamentos
 - ✦ Cada abstração deve usar os recursos disponíveis nos níveis inferiores.
 - ✦ Independência entre os níveis
 - Mudanças em um nível não devem afetar as operações dos níveis superiores (ex: família de computadores).

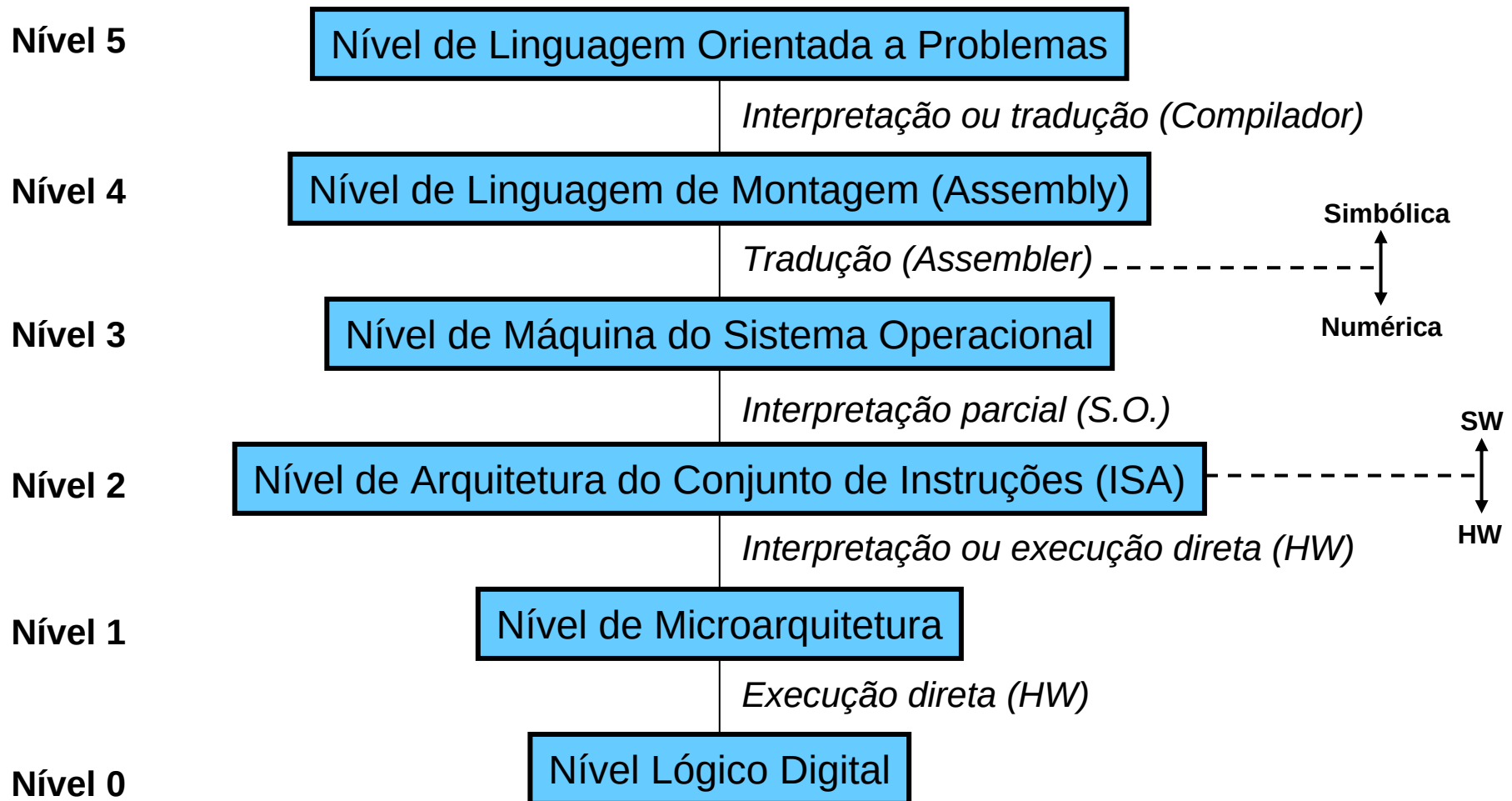
Organização Estruturada (Multi-níveis)

★ Ponto de vista de programação:

- ✓ Computador é uma máquina para resolução de problemas através da execução de programas.
 - ✗ Programa = seqüência de instruções.
- ✓ Instruções primitivas de um computador (linguagem de máquina) representam operações extremamente simples.
 - ✗ Coerentes com os requisitos de desempenho, de complexidade e de custo do projeto da máquina.
 - ✗ São numéricas, de difícil entendimento e utilização.
- ✓ **Desafio:** Aproximar a linguagem de máquina (L_0) e a linguagem conveniente ao programador (L_n).
- ✓ **Solução:** Criar uma série de computadores hipotéticos (máquinas reais) com linguagens de máquina mais próximas de L_n .
 - ✗ A linguagem L_i deve ser **traduzida** para L_{i-1} ou **interpretada** por um programa escrito em L_{i-1} .
 - ✗ Linguagens adjacentes não devem ser muito diferentes.

Organização Estruturada (Multi-níveis)

Exemplo de máquina multi-níveis (Tanenbaum):



Nível Lógico Digital (Nível 0)

- ✦ É o verdadeiro Hardware do computador.
- ✦ Seus principais componentes são as **portas lógicas**.
 - ✓ Apesar da natureza analógica, podem ser modeladas com precisão como dispositivos digitais.
 - ✓ Produz como saída alguma função simples a partir de suas entradas (ex: AND, OR, NOT).
 - ✓ São agrupadas em circuitos integrados (chips):
 - ✦ Combinacionais (multiplexadores, decodificadores, etc.)
 - ✦ Aritméticos (deslocadores, somadores, etc.)
 - ✦ Memórias de 1 Bit.
 - ✓ A combinação de CIs formam estruturas mais complexas
 - ✦ Ex: registradores, ULA, memórias, etc.

Nível de Microarquitetura (Nível 1)

- ★ Sua função é **interpretar e executar** as instruções do nível 2 (ISA).
 - ✓ **Por microprograma** (interpretação)
 - ✗ Instruções mais complexas
 - ✗ Hardware mais simples $\Rightarrow$ mais barato
 - ✓ **Por hardware** (execução direta)
 - ✗ Melhor desempenho
 - ✗ Hardware mais complexo $\Rightarrow$ mais caro
 - ✓ **Abordagem híbrida**
- ★ Controla a operação do caminho de dados:
 - ✓ 8 a 32 registradores
 - ✓ Unidade Lógica e Aritmética
 - ✓ Barramento de conexão.

Nível ISA (Nível 2)

- ★ É a interface entre o Software e o Hardware
 - ✓ Sua linguagem deve ser conhecida tanto pelos compiladores quanto pelo Hardware
 - ✓ O projeto de uma boa ISA considera:
 - ✗ Facilidade de implementação em HW (atual e futura).
 - ✗ Facilidade na geração de um bom código ISA.
- ★ Possui um documento formal de definição
 - ✓ Visa padronização entre diferentes máquinas/projetistas.
- ★ Provê 2 modos de execução:
 - ✓ **Kernel**: deve executar o S.O. e permite a execução de todas as instruções.
 - ✓ **Usuário**: deve executar progs. aplicativos e não permite instruções sensíveis (ex: manipulação direta da CACHE).
- ★ Permite o acesso por variáveis (indireto) a alguns registradores.

Nível de Máquina do S.O. (Nível 3)

- ✦ É geralmente um nível híbrido formado por:
 - ✓ Instruções que também pertencem ao nível ISA.
 - ✖ Executadas por microprogramas ou diretamente em HW.
 - ✓ Chamadas de sistema (instruções adicionais).
 - ✖ Executadas por um interpretador que roda no nível 2 (S.O.).
- ✦ Este nível provê:
 - ✓ Organização diferente da memória (particionamento, paginação e memória virtual).
 - ✓ Capacidade de execução de 2 ou mais programas simultâneos.
 - ✓ Etc.

Nível de Linguagem de Montagem (Nível 4)

- ✦ É uma forma simbólica para uma das linguagens subjacentes.
- ✦ Sua linguagem contém palavras e/ou abreviações
 - ✓ Melhor entendimento para os programadores.
- ✦ Um código em assembly é traduzido para os níveis subjacentes por um programa denominado **assembler**.

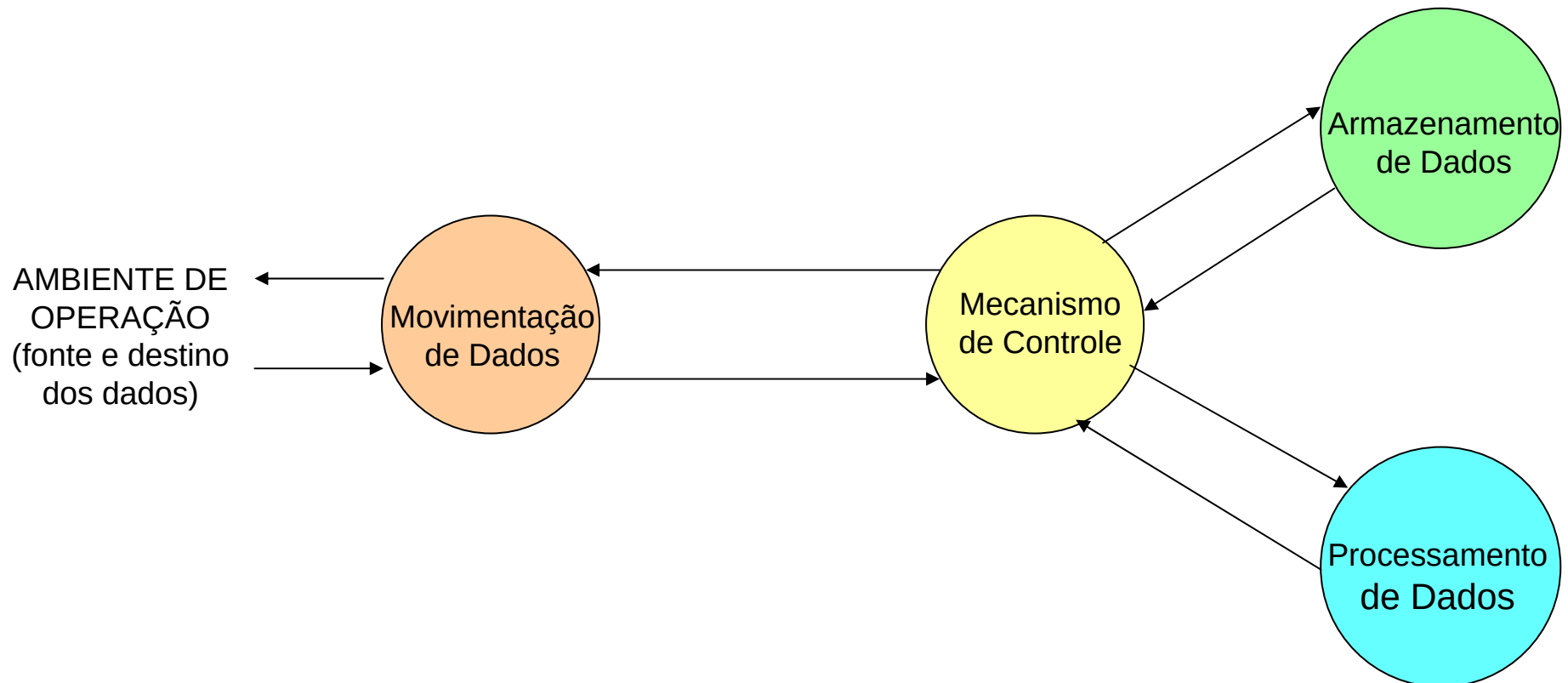
Nível de Linguagem Orientada a Problemas (Nível 5)

- ✦ É o nível mais utilizado pelos programadores de aplicações.
 - ✓ Sua linguagem é de fácil entendimento (linguagem de alto nível).
- ✦ Um código do nível 5 costuma ser traduzido ou interpretado para os níveis 3 ou 4.
 - ✓ Compilador = programa tradutor.

Estrutura e Função

- ★ **Revisão:** A natureza hierárquica dos computadores é essencial para sua definição e projeto.
 - ✓ Projetista lida com um nível do sistema por vez (subsistema).
- ★ Em cada nível, o projetista considera:
 - ✓ **Estrutura:** inter-relacionamento dos componentes.
 - ✓ **Função:** operação de cada componente como parte da estrutura.
- ★ Tanto a estrutura quanto a função de um computador são **simples** em sua essência.

Visão Funcional



Visão Funcional

✦ Funções básicas de um computador:

✓ **Processamento de dados**

- ✦ Operações lógicas e aritméticas

✓ **Armazenamento de dados:**

- ✦ Dados e instruções

✓ **Movimentação de dados:** entre componentes internos e com o meio externo.

- ✦ **E/S:** movimentação para periféricos
- ✦ **Comunicação de dados:** movimentação para dispositivos remotos

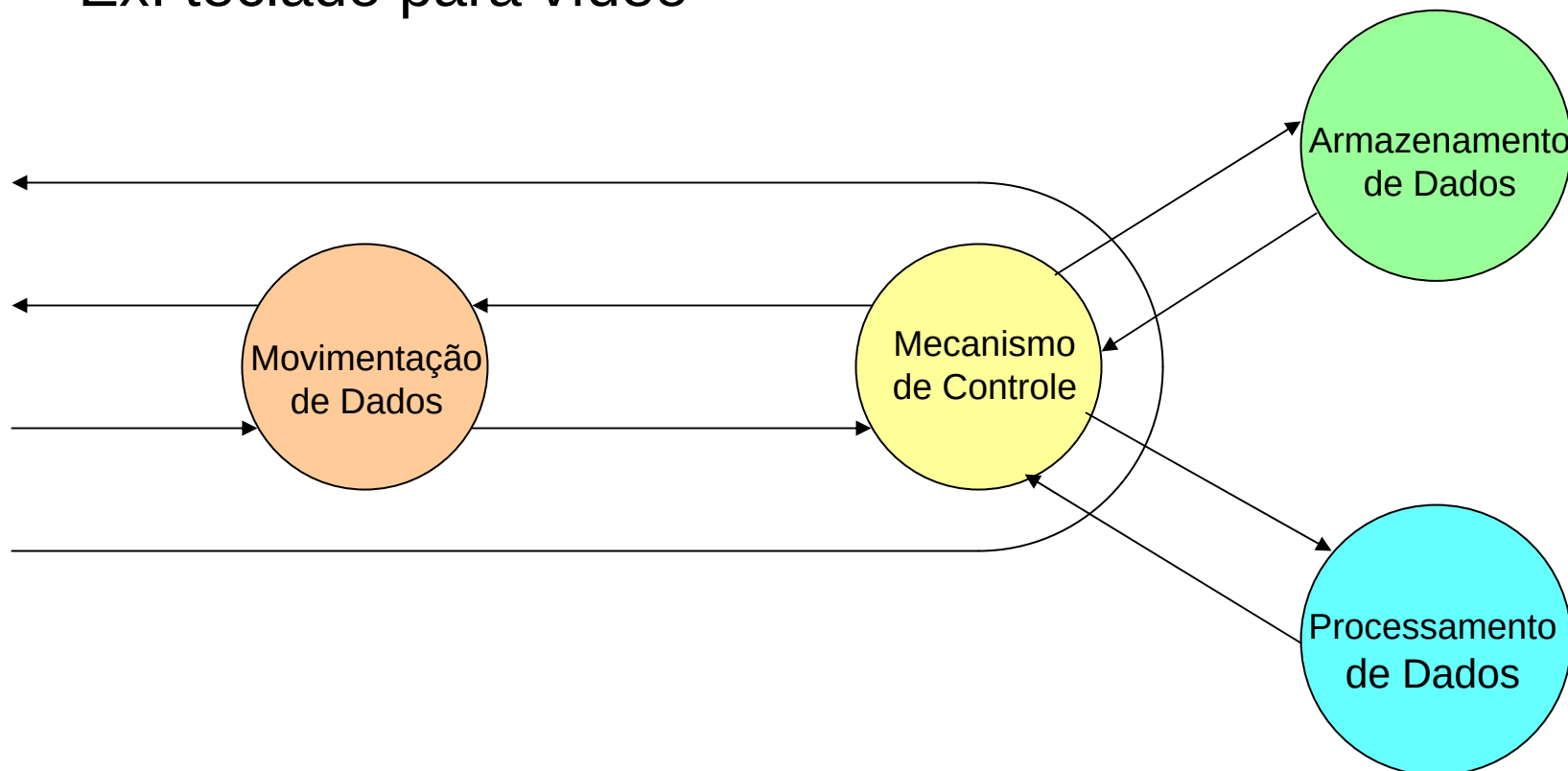
✓ **Controle**

- ✦ Gerenciamento e sequenciamento das demais funções
- ✦ Controle dos componentes

Operações

★ Movimentação de dados

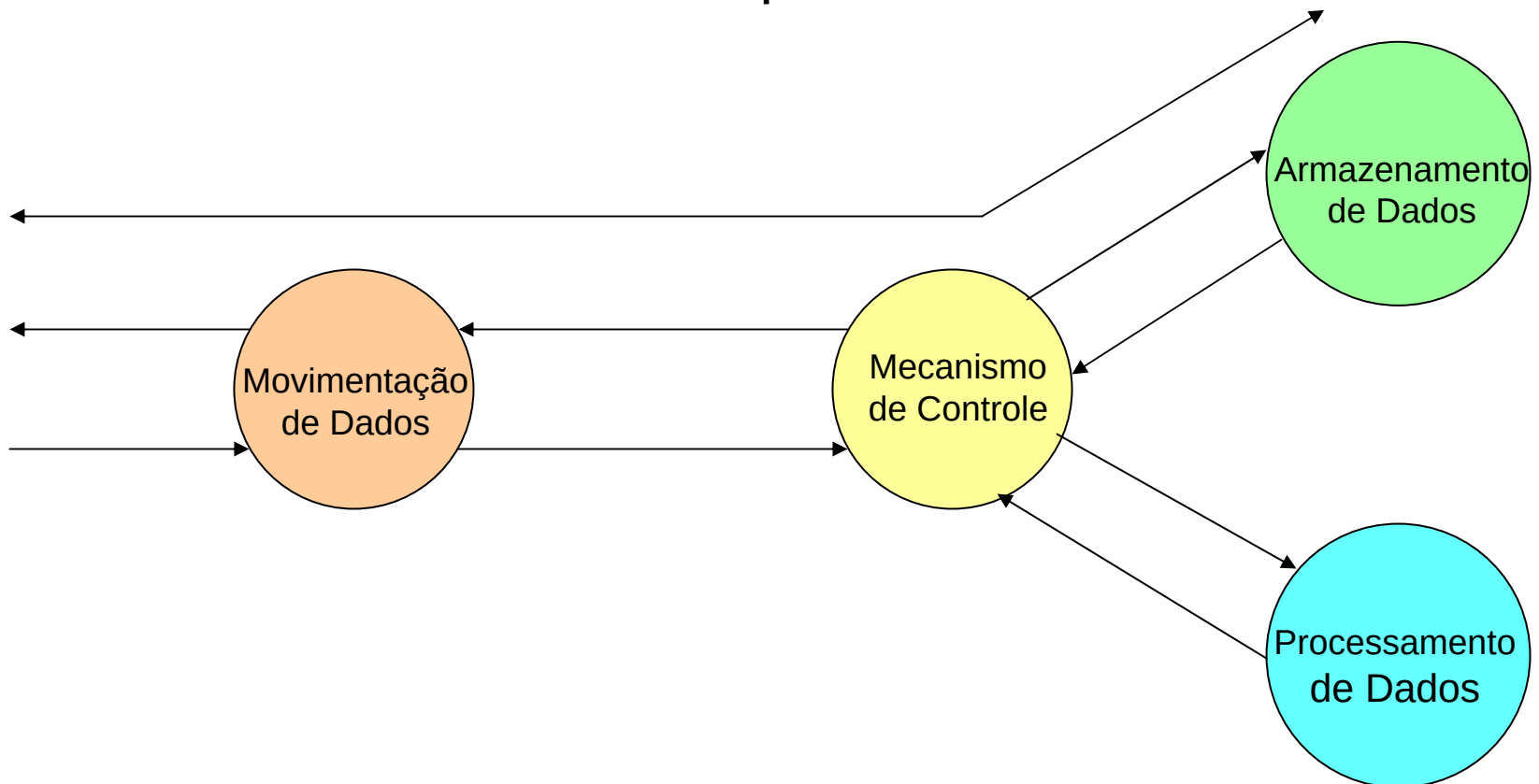
✓ Ex: teclado para vídeo



Operações

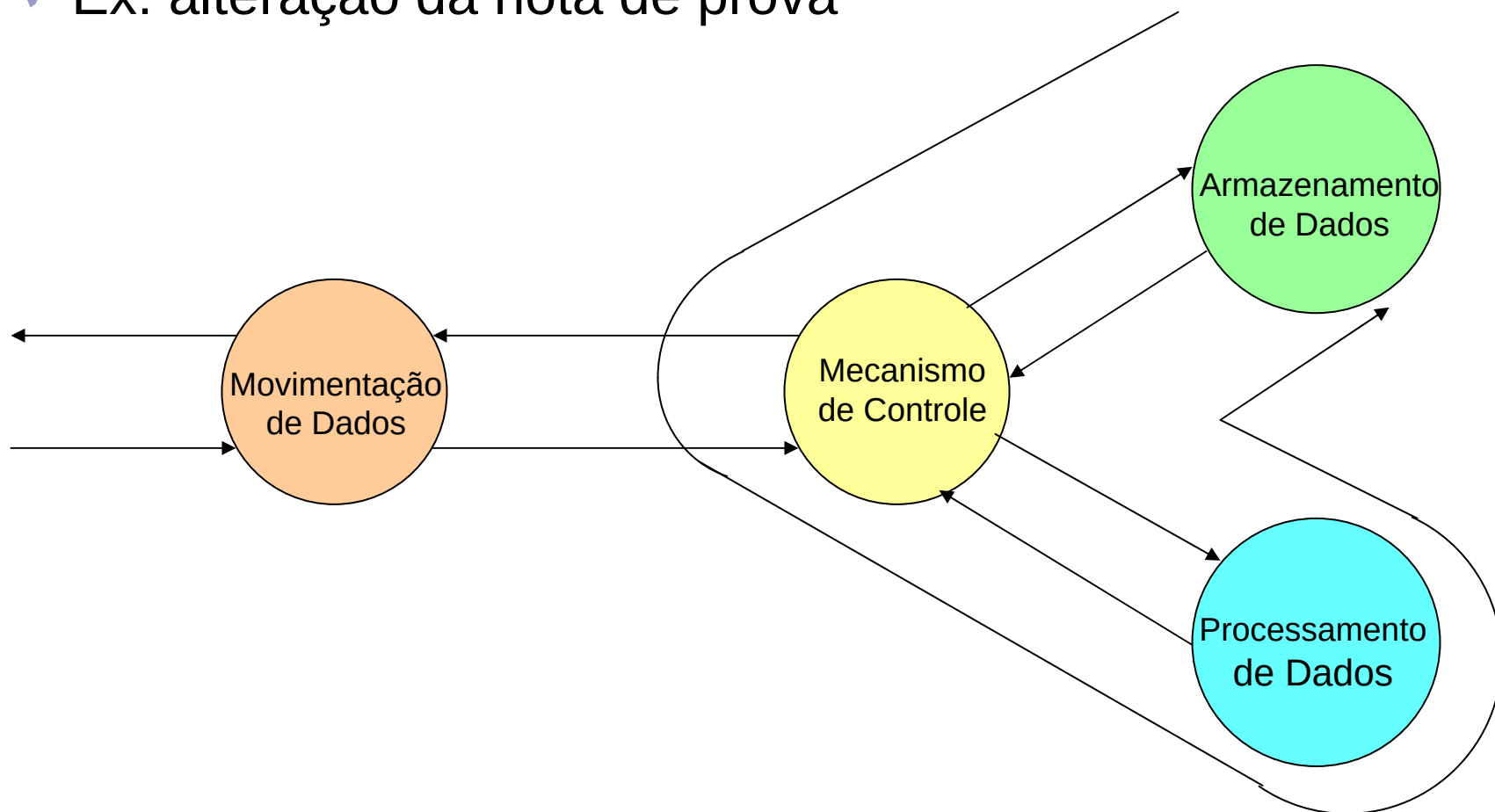
★ Armazenamento

✓ Ex: Download da internet para o disco



Operações

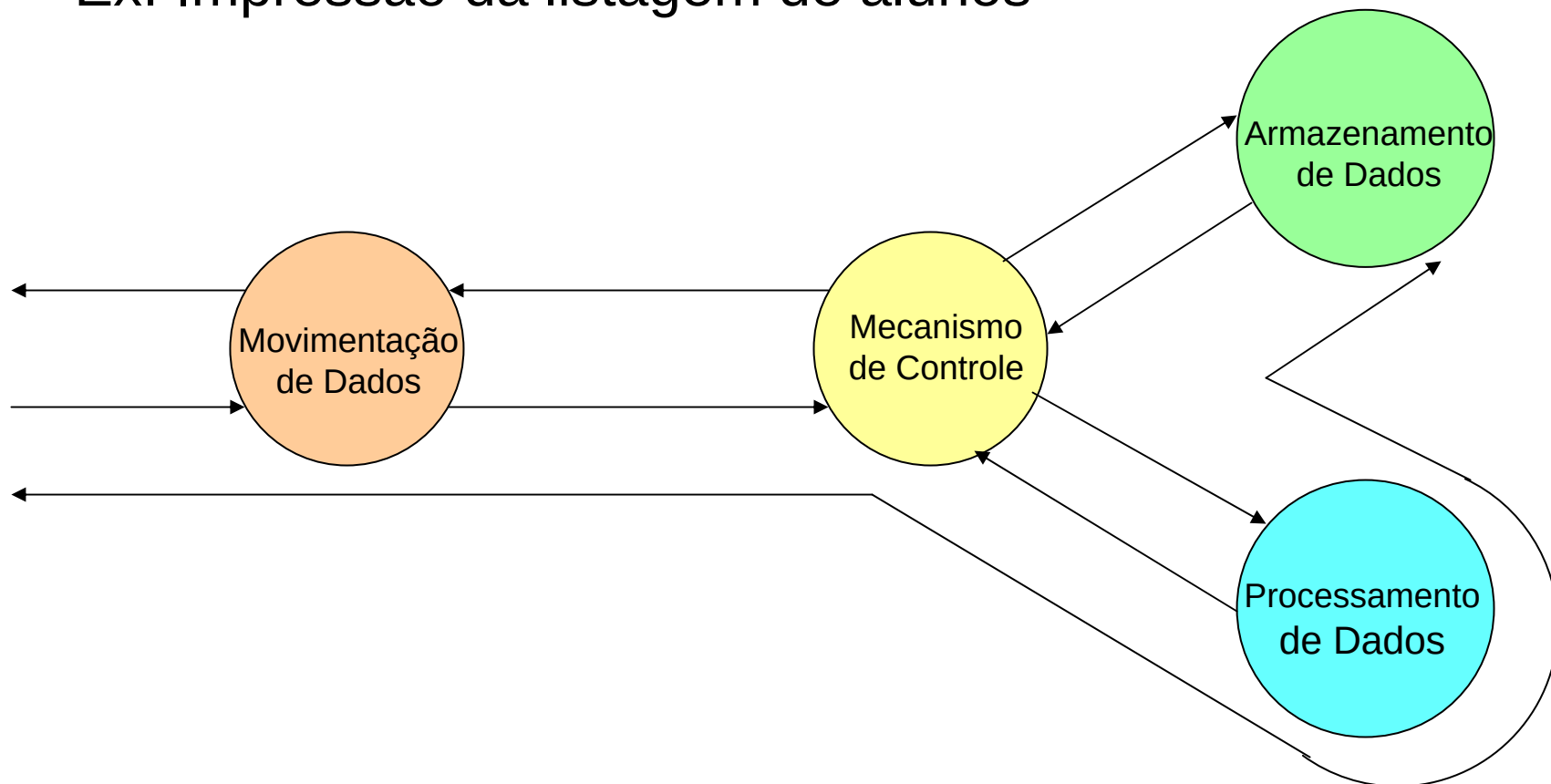
- ✦ Processamento sobre dados armazenados (*de/para memória*)
 - ✓ Ex: alteração da nota de prova



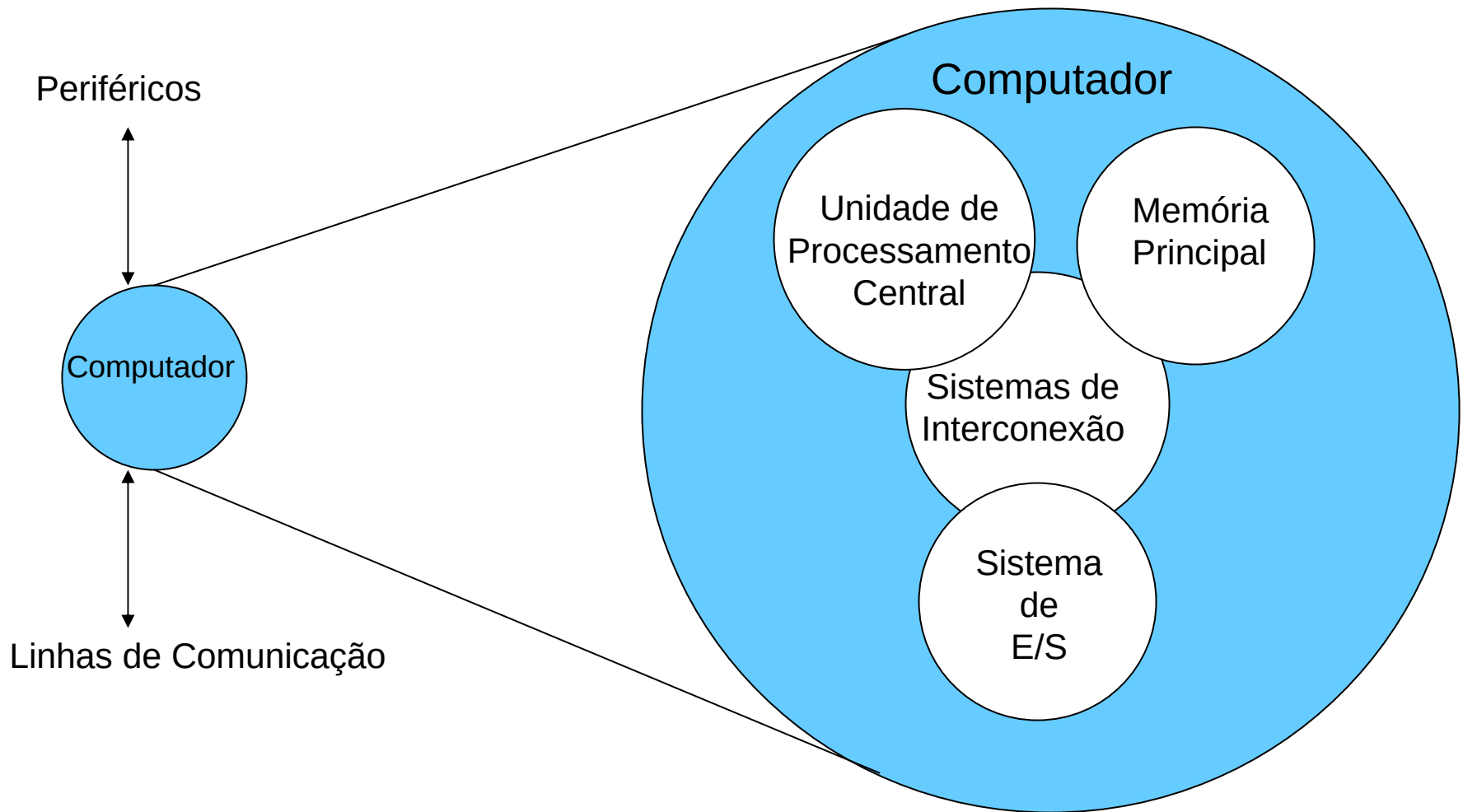
Operações

★ Processamento da memória para E/S

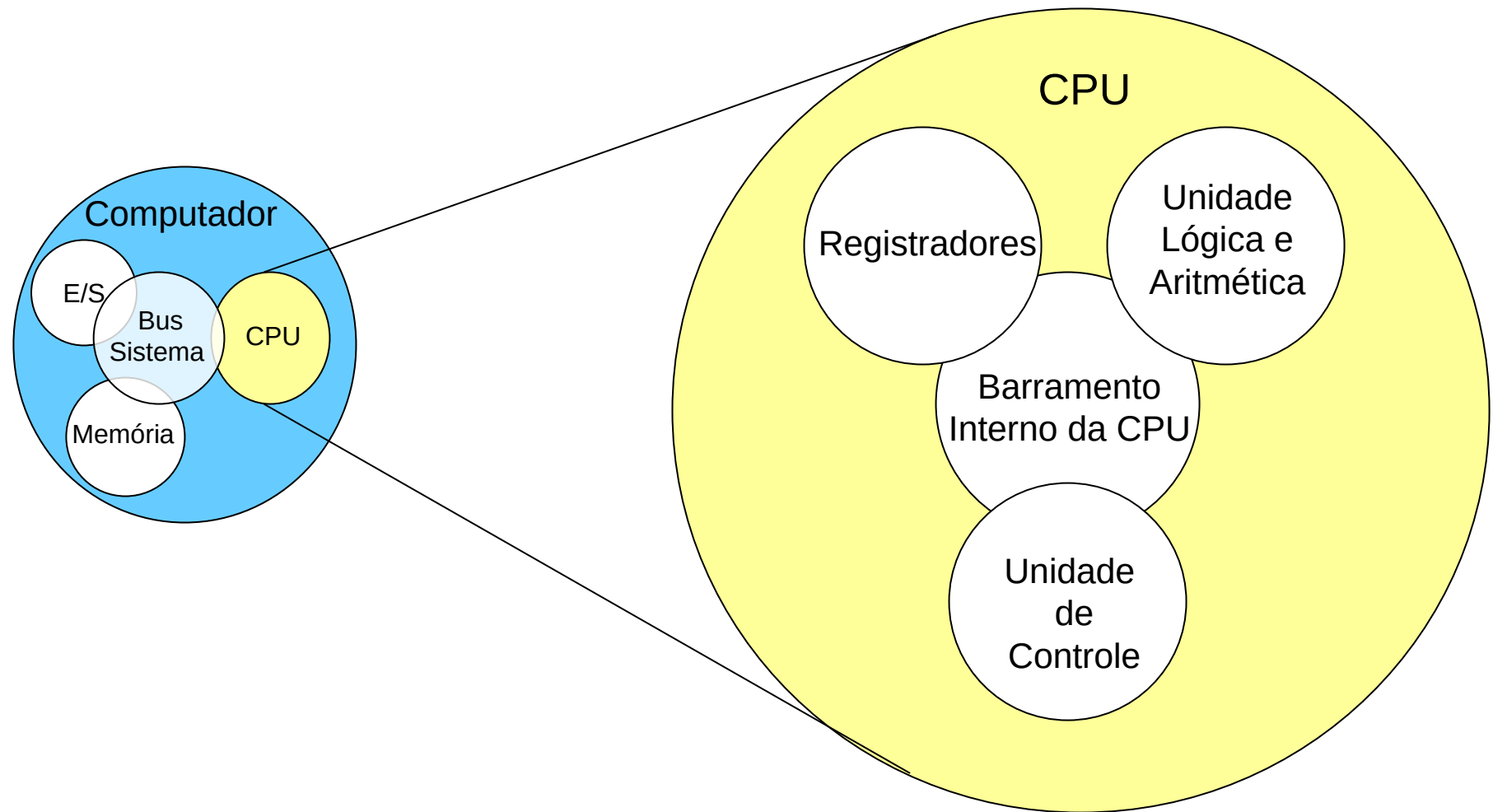
✓ Ex: Impressão da listagem de alunos



Visão Estrutural



Visão Estrutural da CPU



Processadores: Conceito de Programação

★ Programação em **hardware**:

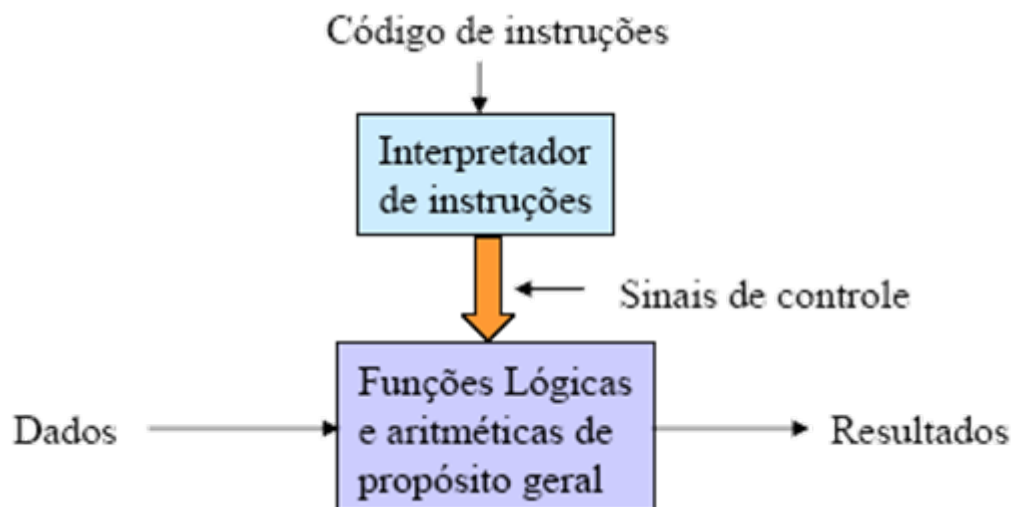
- ✓ Hardware dedicado (realiza uma única tarefa).
- ✓ Programação através da **configuração dos componentes** lógicos para a execução da aplicação.
 - ✗ Conexão dos componentes do sistema.
 - ✗ Programa **hardwired**.
- ✓ Método inflexível (modificação através da reestruturação do hardware).



Processadores: Conceito de Programação

✦ Programação por **software**:

- ✓ Hardware de propósito geral (realizam tarefas distintas).
- ✓ Programação através de um **conjunto de sinais de controle** gerados para a execução da aplicação.
 - ✦ Seqüenciamento e envio de sinais de controle.
- ✓ Necessidade de um **interpretador de instruções**.
 - ✦ Converte cada instrução nos sinais de controle correspondentes.
- ✓ Método flexível (mudanças através de um novo conj. de instruções).



Visão Estrutural da Unidade de Controle

